Military College of Signals
National University of Science and Technology (NUST)
Department of Computer Software Engineering

PBL – PHASE 2

Course Code: SE-314

Course Title: Software Construction

Submitted By:
1. Hamza Haider (454989)
2. Hassnain Bin Nisar (456079)
3. Maaz Ahmad (469539)
4. Ahtisham Arif (460560)
5. GC Zaeem Sajid (486850)
6. GC Tayyab (486856)

Submitted To: Lec Fawad Khan

Course: BESE 29-A

Date: 4th Decemeber, 25

Contents

1. **Objective of the Phase** 3
2. **Scope of the Design Report** 3
3. **Tools and Technology Stack** 3
   - **3.1. Backend Technologies (The Core Engine)** 3
   - **3.2. Frontend Technologies (The User Interface)** 4
   - **3.3. External APIs & Services (The Intelligence)** 5
   - **3.4. Development Tools (The Workflow)** 6
4. **Use Case Diagram** 7
   - **4.1 Multi-User Account Management (Registration & Login)** 7
   - **4.2 Personalized Dashboard & Visualization** 8
   - **4.3 Dynamic Location Updates** 9
   - **4.4 Integrated Chatbot for Suggestions** 10
   - **4.5 Gmail-Based Alerts (System Triggered)** 11
5. **Class Diagram** 12
6. **Sequence Diagram** 13
7. **Deployment Diagram** 14

1. Objective of the Phase

The goal of this phase is to convert the proposed solution into a well-structured software design using solid engineering principles and modeling techniques. It defines the tools, frameworks, and technologies needed to build the aqi-monitoring-system. The focus is to ensure every system component — from the machine learning model to the web interface — is designed with scalable, efficient, and appropriate technologies.

2. Scope of the Design Report

This report concentrates on identifying and describing the full range of technologies and frameworks needed to build the system. It details the front-end and back-end stacks, programming languages, APIs, database solutions, machine learning libraries, authentication components, and deployment tools that will work together to implement the aqi-monitoring-system.

3. Tools and Technology Stack

The system is architected as a modern, decoupled full-stack application. It utilizes a robust Java Spring Boot backend for business logic and data processing, paired with a responsive React frontend for user interaction. The entire stack is container-ready and designed for scalability.

3.1. Backend Technologies (The Core Engine)

- **Language: Java 17 (LTS)**
    - **Role:** The primary programming language for the backend.
    - **Why:** Chosen for its robust type safety, massive ecosystem, and Long-Term Support (LTS) stability. It powers the Spring framework efficiently.
- **Framework: Spring Boot 3.5.7**
    - **Role:** The backbone of the server-side application.
    - **Why:** It provides "convention over configuration," allowing rapid development of RESTful APIs. Key modules used include:
        - spring-boot-starter-web: For building the REST API endpoints.
        - spring-boot-starter-data-jpa: For database interaction via Hibernate.
        - spring-boot-starter-security: For stateless authentication and authorization.

        - spring-boot-starter-mail: For SMTP email integration.
- **Security: Spring Security 6 + JWT (JSON Web Tokens)**
    - **Role:** Handles Authentication (AuthN) and Authorization (AuthZ).
    - **Why:** We implemented a **stateless** security model. Instead of server-side sessions, the system issues a signed JWT upon login (JJWT library). This token is validated via a custom filter chain (JwtAuthenticationFilter) on every request, making the backend scalable and secure.
- **Database: PostgreSQL 16**
    - **Role:** The primary relational database management system (RDBMS).
    - **Why:** An open-source, ACID-compliant database chosen for its reliability and support for complex queries. It stores user profiles, preferences, and the critical time-series data for AQI readings.
- **ORM: Hibernate (via Spring Data JPA)**
    - **Role:** Object-Relational Mapping.
    - **Why:** Eliminates the need for boilerplate JDBC code. It allows us to interact with the database using Java objects (Entities) and simple interface methods (e.g., findFirstByCityIgnoreCaseOrderByTimestampDesc), handling complex SQL generation automatically.

3.2. Frontend Technologies (The User Interface)

- **Library: React 18**
    - **Role:** The component-based library for building the user interface.
    - **Why:** Industry standard for SPAs (Single Page Applications). Its virtual DOM ensures high performance, and its component model allows for modular development (e.g., reusable Navbar, Chart components).
- **Build Tool: Vite**
    - **Role:** The build tool and development server.
    - **Why:** Replaces the slower Create React App (CRA). It offers instant server start and lightning-fast Hot Module Replacement (HMR) using native ES modules.
- **Language: TypeScript**
    - **Role:** A typed superset of JavaScript.

    - **Why:** Adds static typing to the frontend. This was critical for syncing data structures with the backend (e.g., ensuring the AqiDataDto from Java matches the AqiData interface in React), preventing runtime errors.
- **State Management: TanStack Query (React Query v5)**
    - **Role:** Manages server state (fetching, caching, synchronizing).
    - **Why:** Replaces complex useEffect logic. It handles background data polling (refreshing AQI every 15 minutes automatically) and loading/error states seamlessly.
- **Styling: Tailwind CSS**
    - **Role:** Utility-first CSS framework.
    - **Why:** Allows for rapid styling directly in the markup without writing custom CSS files. It enabled the dynamic color coding of AQI cards (e.g., bg-red-500 for unhealthy air) effortlessly.
- **Visualization: Recharts**
    - **Role:** Charting library.
    - **Why:** Built specifically for React. It was used to render the **Area Chart** on the History page and the **Line Chart** for forecasts, translating raw JSON data into interactive visual trends.
- **Mapping: Leaflet & React-Leaflet**
    - **Role:** Interactive maps.
    - **Why:** Used in the Admin Dashboard to plot city markers geographically. It is lightweight and open-source, avoiding the costs/complexity of Google Maps.

3.3. External APIs & Services (The Intelligence)

- **Open-Meteo Air Quality API:**
    - **Role:** The primary source of environmental data.
    - **Why:** Provides granular data ($PM_{2.5}$, PM_{10}, NO_2) and 4-day forecasts for specific coordinates. Crucially, it offers a generous free tier (10k calls/day) suitable for our polling architecture.
- **Groq Cloud API (Llama-3 Model):**
    - **Role:** The Generative AI engine for the Chatbot.

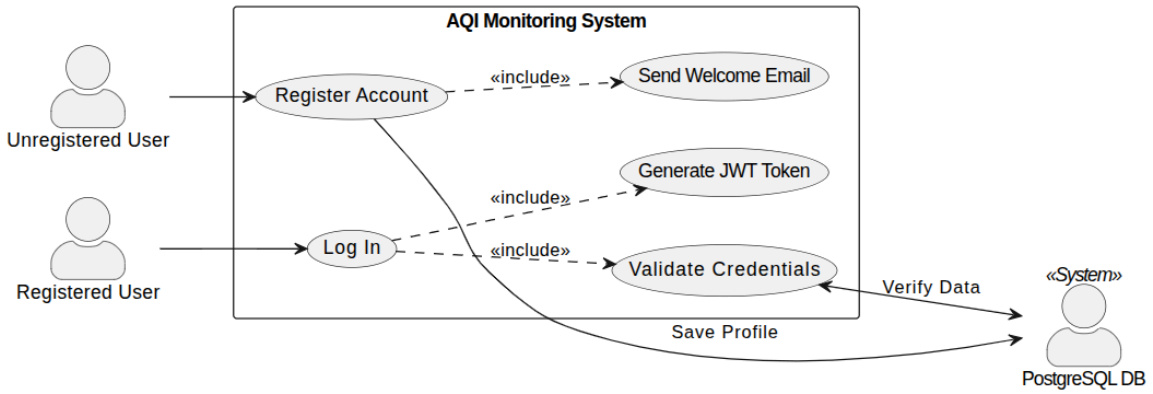
    - **Why:** Chosen for its ultra-low latency inference. It powers the RAG (Retrieval-Augmented Generation) pipeline, taking live AQI context and generating personalized health advice in real-time.
- **Gmail SMTP Server:**
    - **Role:** The email delivery vehicle.
    - **Why:** Used to dispatch Welcome Emails and automated AQI Alerts to users.

3.4. Development Tools (The Workflow)

- **IDEs:**
    - **IntelliJ IDEA (Ultimate/Community):** Used for Backend development (Java/Spring).
    - **VS Code:** Used for Frontend development (React/TS).
- **API Testing: Swagger UI (SpringDoc OpenAPI)**
    - **Role:** Live API documentation.
    - **Why:** Automatically generated a web interface (/swagger-ui.html) for testing backend endpoints without writing frontend code.
- **Configuration Management: Dotenv (Java & JS)**
    - **Role:** Environment variable management.
    - **Why:** Loads sensitive secrets (API Keys, DB Passwords) from a .env file, ensuring they are never hardcoded into the source code (Security Best Practice).
- **Containerization: Docker**
    - **Role:** Deployment packaging.
    - **Why:** Used to package the application into a container image (eclipse-temurin:17-jdk-alpine), ensuring it runs consistently across different environments (Dev, Test, Production).

4. Use Case Diagram

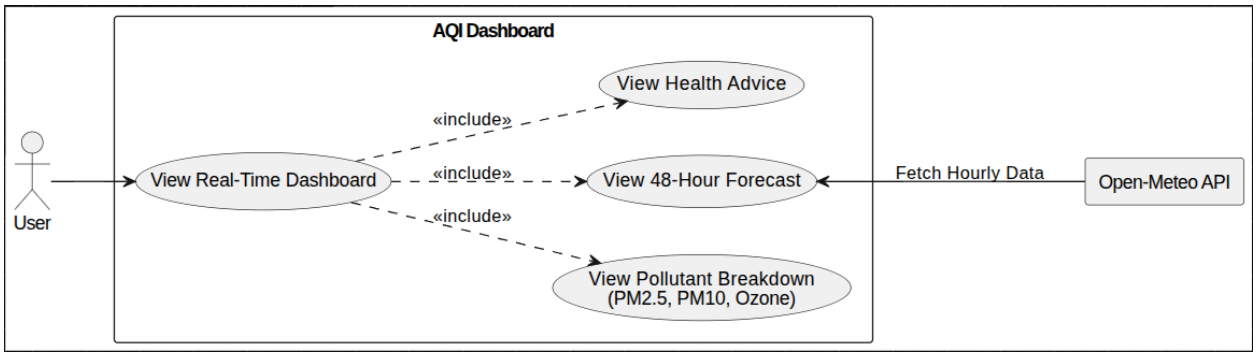
4.1 Multi-User Account Management (Registration & Login)



Unique use case ID	**UC01**
Actor(s)	Unregistered User, Registered User
Description	A user creates a new account to access the system or logs into an existing account to view their personalized dashboard.
Precondition	The user has access to the application URL and a valid email address.
Flow of Events	1. User navigates to the Login/Register page. 2. The user enters email and password. 3. System validates the input format (e.g., valid email regex, password length). 4. System verifies credentials against the PostgreSQL database. 5. System generates and issues a JWT (JSON Web Token). 6. System redirects the user to the Dashboard.
Post conditions	The user is authenticated, the session is active via JWT, and the user is redirected to the protected Dashboard route.
Alternative flows and exceptions	1. **Invalid Credentials:** If the email/password does not match, the system displays a "401 Unauthorized" error toast. 2. **Email Already Exists:** During registration, if the email is taken, the system prevents creation and notifies the user.
No behavioral requirements	No
Issues	None

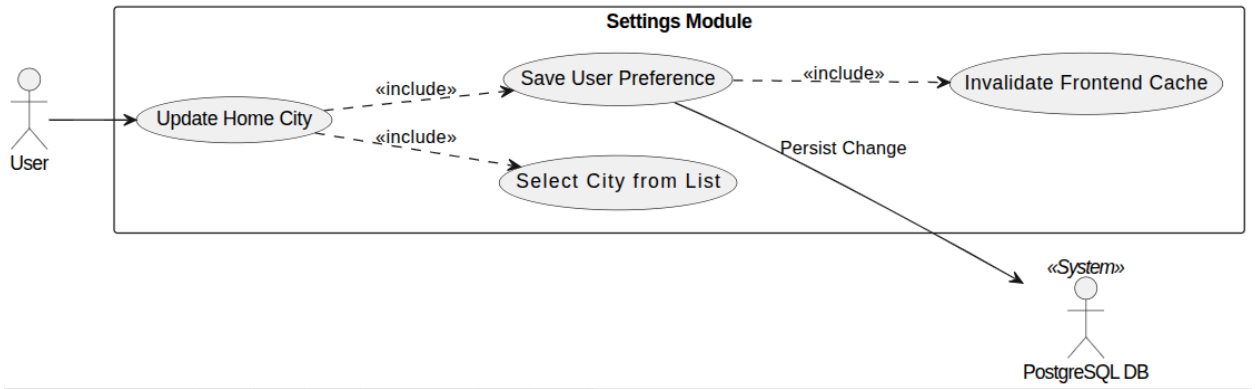
Extend/include	Includes Sending Welcome Email

4.2 Personalized Dashboard & Visualization



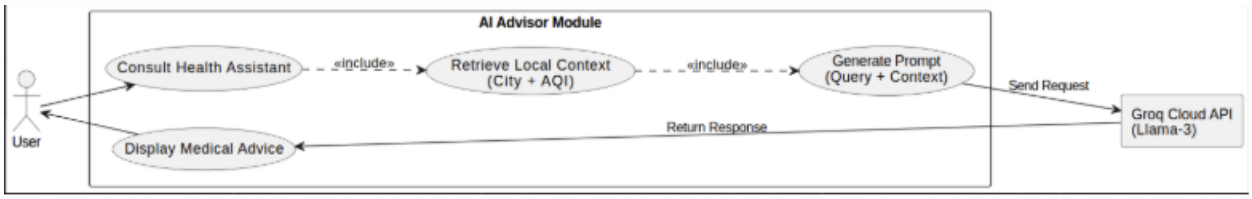
Unique use case ID	**UC02**
Actor(s)	User
Description	The user views real-time air quality data, pollutant breakdowns, and forecasts for their selected home city.
Precondition	User must be logged in and have a valid "Home City" selected in settings.
Flow of Events	1.User accesses the Dashboard page. 2. System retrieves the user's preferred city from the profile. 3. System fetches the latest AQI data point from the database. 4. System displays the current AQI with color-coded severity (Green to Maroon). 5. System renders the pollutant breakdown (PM2.5, PM10, Ozone) and the 48-hour forecast chart.
Post conditions	The user is presented with the latest environmental data visualization.
Alternative flows and exceptions	1. **No Data Available:** If the polling service hasn't run yet, the system displays a "Loading" state or "No data available" message. 2. **No City Selected:** If the user hasn't set a city, the system automatically redirects them to the Settings page (UC03).
No behavioral requirements	No
Issues	None
Extend/include	None

4.3 Dynamic Location Updates



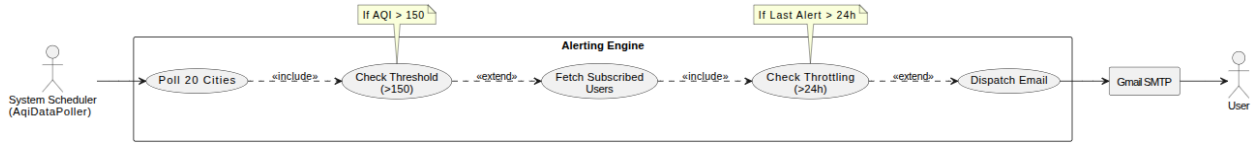
Unique use case ID	**UC03**
Actor(s)	User
Description	The user updates their preferred location to monitor air quality in a different city.
Precondition	User must be logged in.
Flow of Events	1. User navigates to the Settings page. 2. System displays a dropdown list of supported cities (e.g., Lahore, Karachi). 3. User selects a new city and clicks "Save". 4. System sends a PUT request to update the user profile in the database. 5. System confirms the update via a success notification.
Post conditions	The user's profile is updated in the database, and the Dashboard immediately reflects data for the new city.
Alternative flows and exceptions	**Network Failure:** If the API call fails, an error toast is displayed, and the setting is not saved.
No behavioral requirements	No
Issues	None
Extend/include	None

4.4 Integrated Chatbot for Suggestions



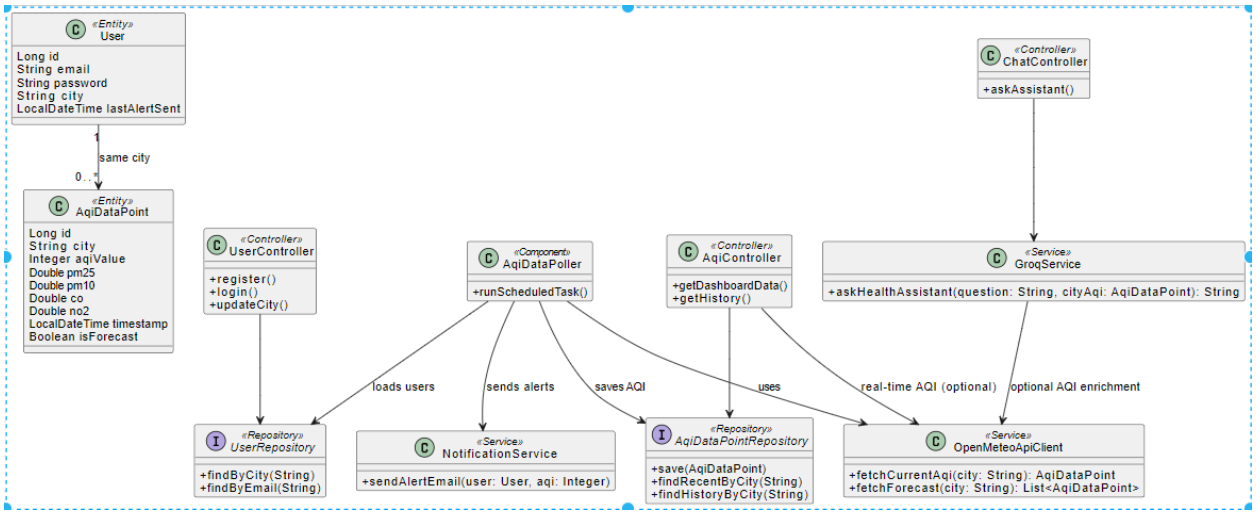
Unique use case ID	**UC04**
Actor(s)	Users
Description	The user interacts with an AI chatbot to receive personalized health advice based on current air quality conditions.
Precondition	User must be logged in and viewing the Chat interface.
Flow of Events	1. User types a natural language query (e.g., "Can I go jogging?"). 2. System retrieves the user's current city and live AQI metrics. 3. System injects this data into a context prompt for the LLM (Groq/Llama-3). 4. AI analyzes the pollution levels against the user's query. 5. System displays the AI's context-aware advice in the chat window.
Post conditions	The user receives actionable health advice relevant to their specific location's current atmosphere.
Alternative flows and exceptions	1. **API Failure:** If the AI service is unreachable, the system returns a fallback message ("I'm having trouble processing that right now"). 2. **Off-Topic Query:** If the user asks about non-health topics, the AI (guided by system prompt) steers the conversation back to air quality.
No behavioral requirements	No
Issues	None
Extend/include	None

4.5 Gmail-Based Alerts (System Triggered)

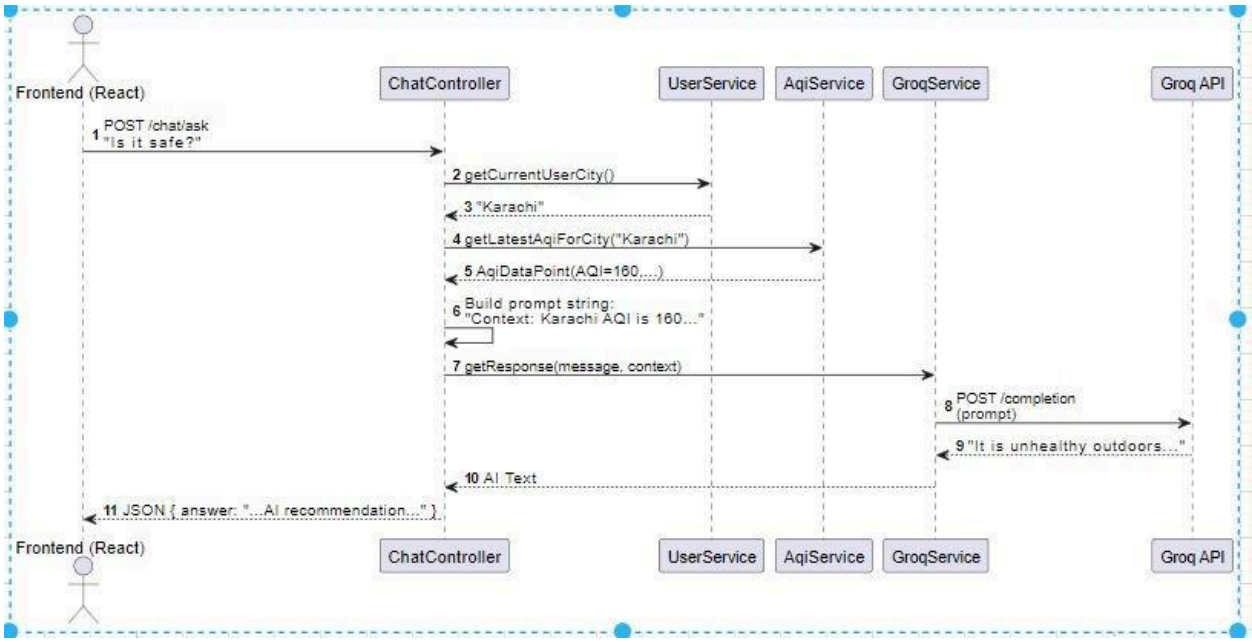


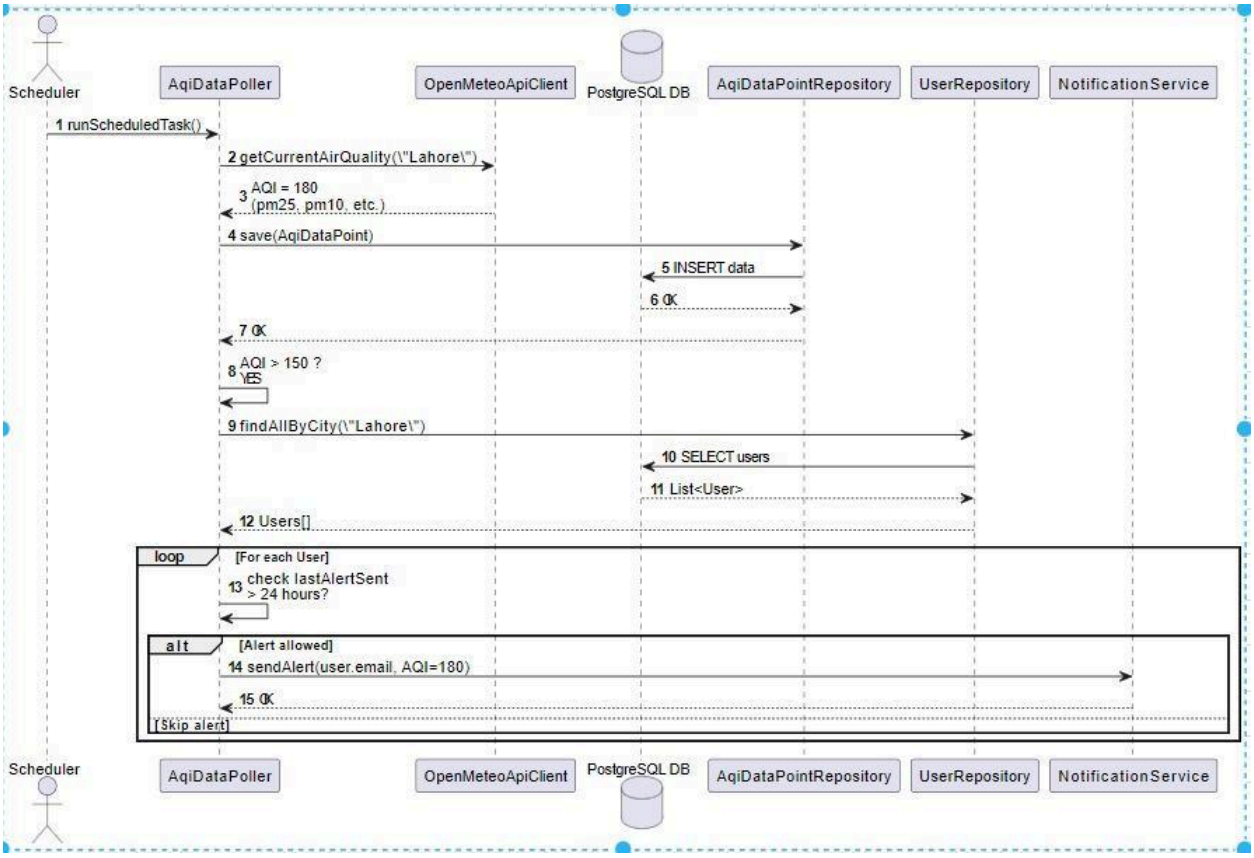
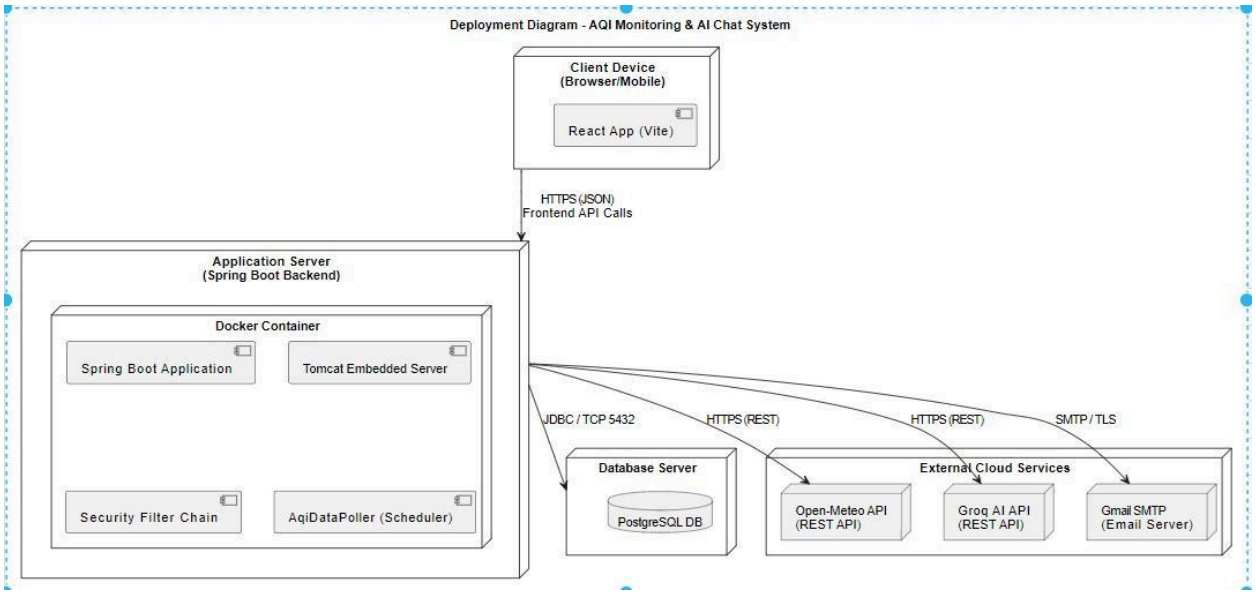
Unique use case ID	**UC05**
Actor(s)	System Scheduler (AqiDataPoller)
Description	The system automatically detects hazardous air quality and sends email notifications to affected users.
Precondition	The backend server must be running, and the Open-Meteo API must be accessible.
Flow of Events	1. System Scheduler triggers the data polling task (every 15 minutes).ser or Medic initiates chat. 2. System fetches fresh AQI data for all cities. 3. System identifies cities where AQI exceeds the "Unhealthy" threshold (150). 4. System retrieves all users subscribed to those specific cities. 5. System checks the lastAlertSent timestamp for each user to prevent spam. 6. System dispatches an alert email via SMTP to eligible users.
Post conditions	Eligible users receive an email warning, and their lastAlertSent timestamp is updated in the database.
Alternative flows and exceptions	**1. Safe Air Quality:** If AQI is below 150, the alert flow terminates immediately. 2. **Throttling Active:** If a user was alerted within the last 24 hours, they are skipped to avoid spammin
No behavioral requirements	No
Issues	None
Extend/include	None

5. Class Diagram



6. Sequence Diagram





7. Deployment Diagram